Making a promo video with code

A step-by-step recipe for a smooth, zooming product video with captions, a voiceover and music, made entirely with free tools on a Mac. Written from building the pixel-pet promo (October 2026).

1 · Storyboard → 2 · Animated web page → 3 · Capture frames → 4 · Encode MP4 / GIF → 5 · Voiceover → 6 · Mix audio → 7 · Check & publish

The idea: instead of a video editor, the whole video is a web page driven by a single function, `seek(t)`, that draws the exact picture for any moment `t`. A script asks Chrome for one screenshot per frame, ffmpeg glues them into a video, and a text-to-speech model reads a timed script on top. Because everything is code, every change is a quick re-render.

Contents

1. Tools to install
2. Plan the storyboard
3. Build the animated page
4. Capture the frames
5. Encode the MP4 and GIF
6. Write the voiceover script
7. Generate the voice
8. Mix voice and music into the video
9. Check and publish
10. Troubleshooting
11. Where the files are

0 Tools to install

Tool	What it does here	Install
Google Chrome	Renders the page and takes the frame screenshots (headless)	Already on most Macs
Node.js 22+	Runs the capture script (uses Node's built-in WebSocket)	`brew install node`
ffmpeg	Frames → MP4, MP4 → GIF, audio mixing	`brew install ffmpeg`
Python 3.11	Runs Kokoro, the open-source voice	`brew install python@3.11`
espeak-ng	Pronunciation data that Kokoro needs	`brew install espeak-ng`

1 Plan the storyboard

Write a table of scenes before any code. Keep each scene to one idea, 3–6 seconds long, with one caption. 30–40 seconds in total is right for LinkedIn or X.

Time	Scene	Camera	Caption
0–4 s	Hero: big sleeping pet, title rises in	Slow push out	(title)
4–10 s	App window; typing wakes the pet	Zoom to prompt	It sleeps… and wakes up to read along
10–13 s	Status line narrates actions	Zoom closer	It narrates what Claude is doing
13–23 s	Helpers hatch; meter climbs; hover card	Pan down, then out	A helper for every subagent / limits at a glance
23–28 s	Grid of 28 colours pops in	Dive into one	28 colours
28–32 s	Cursor clicks the pet; hearts	Still	Click it to pet it
32–37 s	Outro: install commands, link	Settle in	(none)

> Most people watch social video with the sound off, so the captions must tell the whole story on their own. The voiceover is a bonus.

2 Build the animated page

One HTML file at a fixed size (1280×720), no frameworks. Everything that moves is computed from `t` (milliseconds) inside `seek(t)`. No CSS animations or timers, because those can't be paused at an exact frame.

2.1 Easing helpers

```
const clamp = (x, a = 0, b = 1) => Math.min(b, Math.max(a, x))
const lerp  = (a, b, k) => a + (b - a) * k
const io    = k => (k < .5 ? 4*k*k*k : 1 - Math.pow(-2*k + 2, 3) / 2)   // ease in-out: camera moves
const out   = k => 1 - Math.pow(1 - k, 3)                               // ease out: things arriving
const back  = k => { const c = 1.70158, d = k - 1; return 1 + (c+1)*d*d*d + c*d*d }  // springy pop
const prog  = (t, start, ms) => clamp((t - start) / ms)                 // 0→1 over a window
```

2.2 Scenes as layers with zoom-blur transitions

Each scene is an absolutely positioned layer with a start and end time. Arriving, it fades in from slightly blurred; leaving, it grows, blurs and fades. That alone gives the "modern" feel.

```
const FADE = 700
function layer(t, [id, start, end]) {
  const el = document.getElementById(id)
  const k = start === 0 ? 1 : io(prog(t, start - FADE/2, FADE))   // arriving 0→1
  const o = io(prog(t, end - FADE/2, FADE))                       // leaving 0→1
  el.style.opacity = Math.min(k, 1 - o)
  el.style.display = k <= 0 || o >= 1 ? 'none' : ''
  el.style.filter  = o > 0 ? `blur(${o*10}px)` : k < 1 ? `blur(${(1-k)*6}px)` : ''
  return { k, o, el }
}
// in seek(t):  const intro = layer(t, ['intro', 0, 4000])
//              intro.el.style.transform = `scale(${1 + intro.o * .25})`   // grows as it
leaves
```

2.3 A camera for zooms and pans

Put the "app window" in one element and move it with keyframes of `[time, scale, centre x, centre y]`. Repeating a keyframe holds the shot still.

```
const CAM = [[4000, 1.0, 500, 300], [6100, 1.9, 330, 485], [8500, 1.9, 330, 485], [9200,
2.6, 250, 452]]
function camera(t) {
  let i = 0
  while (i < CAM.length - 1 && CAM[i+1][0] <= t) i++
  const a = CAM[i], b = CAM[Math.min(i+1, CAM.length-1)]
  const k = a === b || t <= a[0] ? 0 : io(clamp((t - a[0]) / (b[0] - a[0])))
  return [lerp(a[1], b[1], k), lerp(a[2], b[2], k), lerp(a[3], b[3], k)]
}
// in seek(t):
const [s, cx, cy] = camera(t)
app.style.transformOrigin = '0 0'
app.style.transform = `translate(${640 - cx*s}px, ${360 - cy*s}px) scale(${s})`
```

2.4 Captions, typing and pop-ins

```
// a caption pill rises in, then lifts away
const k = out(prog(t, start, 500)), o = io(prog(t, end - 350, 350))
cap.style.opacity = k * (1 - o)
cap.style.transform = `translate(-50%, ${(1-k)*26 - o*16}px)`

// typing: reveal one character every 48 ms
input.textContent = TEXT.slice(0, Math.floor(clamp((t - TYPE_AT) / 48, 0, TEXT.length)))

// a grid popping in one by one with a spring
cells.forEach((c, i) => { const k = prog(t, START + i*45, 520); c.style.transform =
`scale(${back(k)})` })
```

2.5 Animated SVGs (optional)

If your product already draws SVGs with built-in SMIL animation (like pixel-pet), you can drop them in as they are and seek them too. Each `<svg>` has its own clock:

```
svg.pauseAnimations()
svg.setCurrentTime((t - clipStart) / 1000)   // seconds since this clip appeared
```

Generate those SVGs with your real product code (a small Node script that writes them into `assets.js` as `window.A = {...}`), so the video shows exactly what users will get.

2.6 Expose it to the capture script

```
window.seek = seek
window.DURATION = 37500
// play it live when opened normally, for previewing; #still turns this off for capture
if (location.hash !== '#still') {
  const t0 = performance.now()
  const loop = () => { seek((performance.now() - t0) % DURATION);
requestAnimationFrame(loop) }
  loop()
}
```

> Preview by simply opening the HTML file in Chrome: it loops live. Iterate here until it looks right; only capture when you're happy.

---

3 Capture the frames

`tools/capture.mjs` starts a headless Chrome with a throwaway profile, opens the page, and for every frame calls `seek(t)` and saves a PNG. At device scale 1.5, a 1280×720 page gives 1920×1080 frames.

```
# every frame at 30 fps (37.5 s ≈ 1,125 frames, about 4 minutes)
PAGE=promo.html node tools/capture.mjs frames 30

# just a few moments, to check the layout quickly
PAGE=promo.html node tools/capture.mjs stills 30 1500,8000,15000,30000
```

To eyeball many stills at once, tile them into one contact sheet:

```
ffmpeg -pattern_type glob -i 'stills/t*.png' -vf "scale=640:360,tile=3x3:padding=4" -
frames:v 1 sheet.png
```

How the capture script works, in short:

```
chrome --headless=new --remote-debugging-port=9337 --user-data-dir=<temp dir> about:blank
→ GET http://127.0.0.1:9337/json/list            (find the page's WebSocket URL)
→ Emulation.setDeviceMetricsOverride { width: 1280, height: 720, deviceScaleFactor: 1.5 }
→ Page.navigate { url: file:///…/promo.html#still }
→ for each frame: Runtime.evaluate("seek(t)") then Page.captureScreenshot → frames/00000.png
…
```

4 Encode the MP4 and GIF

MP4 (for LinkedIn, X, YouTube)

```
ffmpeg -framerate 30 -i frames/%05d.png -c:v libx264 -preset slow -crf 18 \
  -pix_fmt yuv420p -movflags +faststart promo.mp4
```

- `-crf 18` is near-lossless; raise it to 23 for a smaller file.
- `yuv420p` and `+faststart` make it play everywhere and start instantly on the web.

GIF (for a GitHub README)

```
ffmpeg -i promo.mp4 -vf "fps=12,scale=640:-1:flags=lanczos,split[a][b];\
[a]palettegen=max_colors=128:stats_mode=full[p];\
[b][p]paletteuse=dither=bayer:bayer_scale=4:diff_mode=rectangle" promo.gif
```

> GitHub shows images up to 10 MB in a README. Moving gradient backgrounds make GIFs huge (the first try was 31 MB at 800 px and 15 fps). 640 px, 12 fps and 128 colours brought it to 8.4 MB. If it is still too big, shorten it or keep the background still.

5 Write the voiceover script

One line per scene, in a tab-separated file: `start seconds ⇥ soft end ⇥ text`. Natural speech runs at about **2.5–3 words per second**, so a 4-second slot fits roughly 10–12 words.

```
0.2     4.3     Meet pixel pet. A tiny friend that lives above your Claude Code prompt.
4.9     9.4     When things are quiet, it sleeps. Start typing, and it wakes up to read
along.
10.3    12.8    It narrates whatever Claude is doing.
13.3    19.2    Every subagent gets its own little helper. And when the job's done, they
wave goodbye.
19.6    23.0    Plus your context and plan limits, always at a glance.
23.8    27.8    Pick from twenty eight colours.
28.9    31.9    And yes, you can pet it.
32.6    37.0    pixel pet. Free and open source, on GitHub.
```

- Write numbers and names the way they should be *said*: "twenty eight", "pixel pet" (not "pixel-pet").
- Start each line a beat *after* its scene begins, so the picture leads and the voice confirms it.
- Leave the last 1–2 seconds silent so the outro can breathe.

6 Generate the voice

Pick one. They are listed from quickest to most natural.

Option A: the Mac's built-in voice (quick, robotic)

```
say -v '?' | grep en_                       # list installed English voices
say -v "Ava (Premium)" -r 180 -o 1.aiff "Meet pixel pet. A tiny friend…"
```

The default voices sound robotic. Better "Premium" voices are free: in System Settings, search for **System voice** (its place in Accessibility moves between macOS versions), open the voice menu → **Manage Voices…**, and download e.g. Ava, Zoe or Evan (Premium). Even these still sound synthetic for marketing.

Option B: Kokoro, open source, near-human (recommended)

Kokoro-82M (Apache 2.0 licence, fine for commercial use) runs fast on Apple Silicon through `mlx-audio`. One-time setup in its own virtual environment, so nothing else on the machine changes:

```
brew install espeak-ng
python3.11 -m venv ~/.venvs/tts
~/.venvs/tts/bin/pip install mlx-audio "misaki[en]"     # also pulls in PyTorch and spaCy

# Kokoro's bundled espeak data path is broken on macOS; point it at Homebrew's copy
export ESPEAK_DATA_PATH=/opt/homebrew/share/espeak-ng-data
export PHONEMIZER_ESPEAK_LIBRARY=$(ls /opt/homebrew/lib/libespeak-ng.*dylib | head -1)
```

Render every line of the script into numbered WAV files (the model, about 330 MB, downloads on first use):

```
mkdir -p voice
i=0
while IFS=$'\t' read -r at end text; do
  i=$((i+1))
  ~/.venvs/tts/bin/python -m mlx_audio.tts.generate --model mlx-community/Kokoro-82M-bf16 \
    --voice af_heart --lang_code a --speed 1.0 --text "$text" \
    --output_path voice --file_prefix $i --audio_format wav --join_audio < /dev/null
done < voiceover-script.tsv
```

Voice	Sound	`--lang_code`
`af_heart`	US female, Kokoro's best-rated voice	`a`
`af_bella`	US female, warmer	`a`
`am_michael`	US male (slower; try `--speed 1.1`)	`a`
`bf_emma`	British female	`b`

Check each clip's length against its slot:

```
for f in voice/*.wav; do echo "$f $(ffprobe -v error -show_entries format=duration -of
csv=p=0 $f)"; done
```

Option C: your own voice, or a clone of it

- **Record it yourself** with QuickTime or your phone, one file per line named `1.wav`, `2.wav`… Then mix exactly as below. For a personal project post this lands best.
- **Chatterbox** (Resemble AI, MIT licence) is the next step up from Kokoro and can speak in your voice from a 5–10 second sample. Only clone voices you have the right to use.
- Avoid F5-TTS and XTTS-v2 for marketing: their licences are non-commercial.

---

7 Mix voice and music into the video

`tools/mix.sh` places each clip at its start time (or just after the previous line ends, if one runs long), evens out the voice loudness, lays the music quietly underneath with a fade in and out, and copies the video stream untouched, so the picture loses no quality.

```
MUSIC=music.wav bash tools/mix.sh voice promo-voiceover.mp4 voiceover-script.tsv promo.mp4
```

The core of it, if you want to do it by hand for one clip:

```
ffmpeg -i promo.mp4 -i voice/1.wav -stream_loop -1 -i music.wav -filter_complex \
 "[1:a]adelay=200|200[v];[2:a]volume=0.18,afade=t=in:d=1.5[m];[v]
[m]amix=inputs=2:normalize=0[a]" \
 -map 0:v -map "[a]" -c:v copy -c:a aac -b:a 192k -shortest out.mp4
```

- `adelay=200|200` starts the clip at 0.2 s (one value per stereo channel, in milliseconds).
- `volume=0.18` keeps the music under the voice. Use 0.3 or more with no voiceover.
- `normalize=0` stops `amix` from quietly halving every input.
- Royalty-free music: Pixabay Music or the YouTube Audio Library.

---

8 Check and publish

```
ffprobe -v error -show_entries stream=codec_type,duration -of csv=p=0 promo-voiceover.mp4
# audio and video, same length
ffmpeg -i promo-voiceover.mp4 -af volumedetect -vn -f null - 2>&1 | grep -E "mean|max"    #
max below 0 dB = no clipping
```

- **Watch and listen to it end to end at least once.** Numbers can't tell you if a voice sounds odd.
- **LinkedIn:** upload the MP4 as a native video (it autoplays muted in the feed; links don't).
- **GitHub:** the GIF goes in the README; link the MP4 for a sharper version.
- **Square feeds (Instagram):** set the page to 1080×1080 and re-capture. Nothing else changes.

9 Troubleshooting

Problem	Fix
Headless Chrome hangs and never exits	Always use a throwaway `--user-data-dir`, wrap runs in a timeout (`perl -e 'alarm 120; exec @ARGV' node …`), and clean up with `pkill -f promo-chrome`.
Elements pile up on top of each other	An `position: absolute` wrapper inside a flex or grid layout drops out of the flow. Make wrappers in flowing layouts `position: relative`.
An SVG animation silently does nothing	In SMIL, `keyTimes` for linear or spline animation must start at 0 and end at 1.
Error mentioning `espeak-ng-data/phontab`	Set `ESPEAK_DATA_PATH` and `PHONEMIZER_ESPEAK_LIBRARY` as in step 6B.
"Kokoro requires the optional 'misaki' package"	`pip install "misaki[en]"` in the same virtual environment.
"bad math expression" in a shell loop	zsh reads `$f[...]` as an array index. Run mixing scripts with `bash`, or write `${f}[...]`.
Voice lines overlap	Shorten the line, start it later, or render faster (`--speed 1.1`). `mix.sh` also pushes a late line back automatically.
GIF too big	Lower fps (10–12), width (480–640) and colours (96–128); cut the length; avoid moving full-screen gradients.
Hover card or cursor in the wrong place after zooming	Convert window coordinates to screen ones with the camera: `screenX = 640 + (x − cx) * s`, `screenY = 360 + (y − cy) * s`.

10 Where the files are

In `~/Projects/Personal/claude-mods/docs/promo/`:

File	What it is
`promo.html` + `assets.js`	The animated page (open in Chrome to preview) and the pet SVGs generated from the plugin's code
`tools/capture.mjs`	Frame capture over the Chrome DevTools Protocol
`tools/mix.sh`	Voice and music mixing
`voiceover-script.tsv`	The timed voiceover script
`pixel-pet-promo.mp4` / `.gif`	The silent video and the README GIF
`pixel-pet-promo-kokoro-*.mp4`	Voiceover versions (Heart, Michael, Emma)

The whole loop for a new video: copy `promo.html`, change the scenes and captions, preview in Chrome, capture stills to check, capture all frames, encode, write the script, render the voice, mix, watch.